

École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes
- Rabat

Etude experimentale de l'impact des niveaux d'observabilite
sur la detection des incidents dans un systeme de sante

Guide Pratique : Observabilite DevOps pour HAPI FHIR

Élèves :
Hajar BRAIDI
Aya Taki
Soukaina Sabbar
Khadija Ait Oufkir

Enseignant :
Prof. Karima Moumane

2 avril 2026

Table des matières

1 Presentation du projet **3**
1.1 Objectif . . . 3
1.2 Les quatre niveaux etudies . . . 3
1.3 Les trois incidents simules . . . 3

2 Prerequis et installation **3**
2.1 Logiciels requis . . . 3
2.2 Cloner le projet . . . 4
2.3 Structure du projet . . . 4

3 Architecture et configuration **4**
3.1 Architecture globale . . . 4
3.2 Le fichier docker-compose.yml . . . 5
3.3 Configuration de Prometheus . . . 8
3.4 Configuration de Promtail . . . 8
3.5 Configuration de Tempo . . . 9
3.6 Configuration des datasources Grafana . . . 10

4 Lancement des niveaux d'observabilite **10**
4.1 Comment lancer chaque niveau . . . 10
4.2 Verification du lancement . . . 11
4.3 Acces aux interfaces . . . 11
4.4 Arreter les conteneurs . . . 12

5 Script d'automatisation des experiences **12**
5.1 Presentation du script . . . 12
5.2 Autoriser l'execution du script . . . 13
5.3 Lancer le script . . . 13
5.4 Ce que le script fait automatiquement . . . 13
5.5 Sortie console typique . . . 14
5.6 Comment sont simules les incidents . . . 15

6 Resultats experimentaux **15**
6.1 Le fichier results.csv . . . 15
6.2 Tableau de synthese des resultats MTTD . . . 16
6.3 Analyse des graphiques . . . 16

7 Visualisation dans Grafana **16**
7.1 Importer le dashboard Grafana . . . 16
7.2 Voir les logs dans Loki . . . 17
7.3 Voir les metriques dans Prometheus . . . 17
7.4 Voir les traces dans Tempo . . . 17

8 Demontrer l'impact en direct – Demo jury **17**

9 Conclusion et recommandations **18**
9.1 Resultats cles 18
9.2 Recommandations pour les equipes DevOps 18

1 Presentation du projet

Ce guide explique comment reproduire integralement le projet **DevOps & Observabilite pour HAPI FHIR**, realise dans le cadre de la Fiche 3 – *Observabilite & DevOps (Health)* de l'ENSIAS 2026. Le code source complet est disponible sur GitHub :

`https://github.com/HajarBraidi/Devops-hapi-fhir-jpaserver`

1.1 Objectif

Mesurer experimentalement l'impact du niveau d'observabilite sur la **detection** et la **resolution** des incidents dans un systeme de sante base sur le standard FHIR.

1.2 Les quatre niveaux etudies

Niveau	Nom	Outils deployes
N0	Sans monitoring	HAPI FHIR + PostgreSQL uniquement
N1	Logs	+ Loki + Promtail + Grafana
N2	Logs + Metriques	+ Prometheus
N3	Observabilite complete	+ Tempo (traces distribuees)

TABLE 1 – Les quatre niveaux d'observabilite etudies

1.3 Les trois incidents simules

#	Incident	Description
1	Mass HTTP 500	100 requetes FHIR invalides envoyees en rafale
2	High Latency	Conteneur HAPI gele via `docker pause` (SIGSTOP)
3	Disk Full	Volume PostgreSQL sature avec un fichier de 500 Mo

TABLE 2 – Les trois incidents simules dans le projet

2 Prerequis et installation

2.1 Logiciels requis

Avant de commencer, installer les logiciels suivants sur votre machine Windows :

— **Docker Desktop** : `https://www.docker.com/products/docker-desktop/`

— **Git** : `https://git-scm.com/downloads`

— **Python 3.x** : `https://www.python.org/downloads/`

— **Visual Studio Code** (recommande) : `https://code.visualstudio.com/`

Verifier les installations dans PowerShell :

```
docker --version
docker compose version
git --version
python --version
```

Listing 1 – Verification des prerequis

2.2 Cloner le projet

```
git clone https://github.com/HajarBraidi/Devops-hapi-fhir-jpaserver.git
cd Devops-hapi-fhir-jpaserver
```

Listing 2 – Clonage du depot GitHub

2.3 Structure du projet

Apres clonage, voici les fichiers importants :

```
Devops-hapi-fhir-jpaserver/
|-- docker-compose.yml          # Orchestration des conteneurs
|-- config/
|   |-- grafana-datasources.yml # Configuration des datasources
|   |-- prometheus.yml          # Configuration du scrape
|   |-- loki-config.yml         # Configuration de Loki
|   |-- promtail-config.yml     # Configuration de Promtail
|   |-- tempo-config.yml        # Configuration de Tempo
|-- experiments/
|   |-- run_experiments.ps1     # Script d'automatisation
|   |-- analyze_results.py      # Analyse des resultats
|   |-- dashboard.html          # Dashboard de visualisation
|-- results.csv                 # Resultats des experiences
```

Listing 3 – Structure des fichiers du projet

3 Architecture et configuration

3.1 Architecture globale

Le projet deploie une architecture micro-services composee de 7 conteneurs Docker orchestres par Docker Compose. La figure ci-dessous illustre les interactions entre les composants.

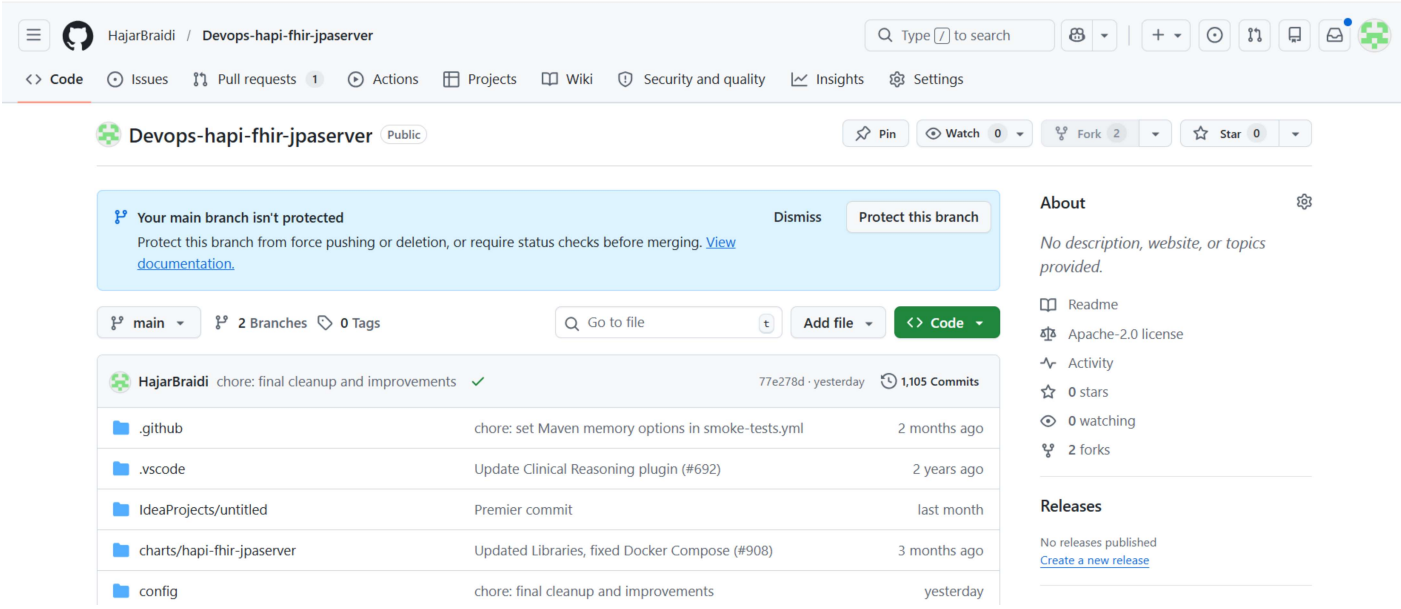


FIGURE 1 – Architecture du système

3.2 Le fichier docker-compose.yml

Le fichier `docker-compose.yml` utilise le mecanisme de **profils** Docker Compose pour activer ou desactiver des groupes de conteneurs. Chaque profil correspond a un niveau d'observabilite.

```yaml
services:
  # Service principal HAPI FHIR (toujours actif)
  hapi-fhir-jpaserver-start:
    image: hapiproject/hapi:latest
    ports:
      - "8080:8080"
    environment:
      JAVA_OPTS: >
        -Dotel.exporter.otlp.endpoint=http://tempo:4317
        -Dotel.service.name=hapi-fhir
        -Dotel.traces.exporter=otlp
    depends_on:
      hapi-fhir-postgres:
        condition: service_healthy

  # Base de donnees PostgreSQL (toujours actif)
  hapi-fhir-postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: hapi
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin
    labels:
      - "logging=promtail"

  # Niveau 1 : Loki (collecte des logs)
  loki:
    image: grafana/loki:2.9.0
    profiles: ["logs"]
    ports:
      - "3100:3100"

  # Niveau 1 : Promtail (agent de collecte)
  promtail:
    image: grafana/promtail:3.0.0
    profiles: ["logs"]
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

Listing 4 – Extrait du fichier docker-compose.yml avec les profils

```yaml
# Niveau 1 : Grafana (visualisation)
grafana:
  image: grafana/grafana:10.0.0
  profiles: ["logs"]
  ports:
    - "3000:3000"

# Niveau 2 : Prometheus (metriques)
prometheus:
  image: prom/prometheus
  profiles: ["metrics"]
  ports:
    - "9091:9090"

# Niveau 3 : Tempo (traces distribuees)
tempo:
  image: grafana/tempo:2.3.0
  profiles: ["traces"]
  ports:
    - "3200:3200"
    - "4317:4317"
```

Listing 5 – Extrait du fichier docker-compose.yml avec les profils

3.3 Configuration de Prometheus

Prometheus utilise le **modele PULL** : il vient interroger HAPI toutes les 15 secondes a l'adresse `/actuator/prometheus` exposee automatiquement par Spring Boot Actuator.

```yaml
# config/prometheus.yml
global:
  scrape_interval: 15s     # Toutes les 15 secondes
  evaluation_interval: 15s

scrape_configs:
  - job_name: "hapi-fhir"
    static_configs:
      - targets: ["hapi-fhir-jpaserver-start:8080"]
    metrics_path: "/actuator/prometheus"
```

Listing 6 – Configuration de Prometheus (prometheus.yml)

3.4 Configuration de Promtail

Promtail surveille les logs de tous les conteneurs Docker en temps reel et les envoie a Loki.

```yaml
# config/promtail-config.yml
server:
  http_listen_port: 9080

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
    relabel_configs:
      - source_labels: [__meta_docker_container_name]
        target_label: container_name
      - source_labels: [__meta_docker_container_label_logging]
        regex: promtail
        action: keep
```

Listing 7 – Configuration de Promtail (promtail-config.yml)

3.5 Configuration de Tempo

Tempo recoit les traces de HAPI via le protocole OTLP sur le port 4317 et les expose a Grafana via son API HTTP.

```yaml
# config/tempo-config.yml
server:
  http_listen_port: 3200

distributor:
  receivers:
    otlp:
      protocols:
        grpc:
          endpoint: 0.0.0.0:4317

storage:
  trace:
    backend: local
    local:
      path: /var/tempo/traces
    wal:
      path: /var/tempo/wal
```

Listing 8 – Configuration de Tempo (tempo-config.yml)

3.6 Configuration des datasources Grafana

```yaml
# config/grafana-datasources.yml
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    url: http://prometheus:9090
    isDefault: true

  - name: Loki
    type: loki
    url: http://loki:3100

  - name: Tempo
    type: tempo
    url: http://tempo:3200
    jsonData:
      tracesToLogs:
        datasourceUid: loki
        filterByTraceID: true
```

Listing 9 – Configuration des datasources Grafana

4 Lancement des niveaux d'observabilite

4.1 Comment lancer chaque niveau

Les commandes suivantes permettent de demarrer l'un des quatre niveaux d'observabilite. Toujours se placer dans le repertoire racine du projet avant de les executer.

```
cd C:\Users\<votre_nom>\Devops-hapi-fhir-jpaserver

# Niveau 0 : HAPI + PostgreSQL uniquement (sans monitoring)
docker compose up hapi-fhir-jpaserver-start hapi-fhir-postgres -d

# Niveau 1 : + Loki + Promtail + Grafana (logs)
docker compose --profile logs up -d

# Niveau 2 : + Prometheus (logs + metriques)
docker compose --profile logs --profile metrics up -d

# Niveau 3 : + Tempo (observabilite complete)
docker compose --profile logs --profile metrics --profile traces up -d
```

Listing 10 – Commandes de lancement par niveau

4.2 Verification du lancement

Apres le lancement, verifier que tous les conteneurs sont actifs :

```
# Verifier l'etat des conteneurs
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Listing 11 – Verification de l'etat des conteneurs

Tous les conteneurs doivent afficher `Up` dans la colonne `STATUS`. Attendre **2 minutes** apres le lancement avant de continuer car HAPI FHIR met environ 90 secondes a demarrer completement.

4.3 Acces aux interfaces

Service	URL	Identifiant	Niveau requis
HAPI FHIR	`http://localhost:8080/fhir/metadata`	–	N0+
Grafana	`http://localhost:3000`	admin/admin	N1+
Prometheus	`http://localhost:9091`	–	N2+
Loki	`http://localhost:3100`	–	N1+
Tempo	`http://localhost:3200`	–	N3

TABLE 3 – URLs d'acces aux interfaces des services

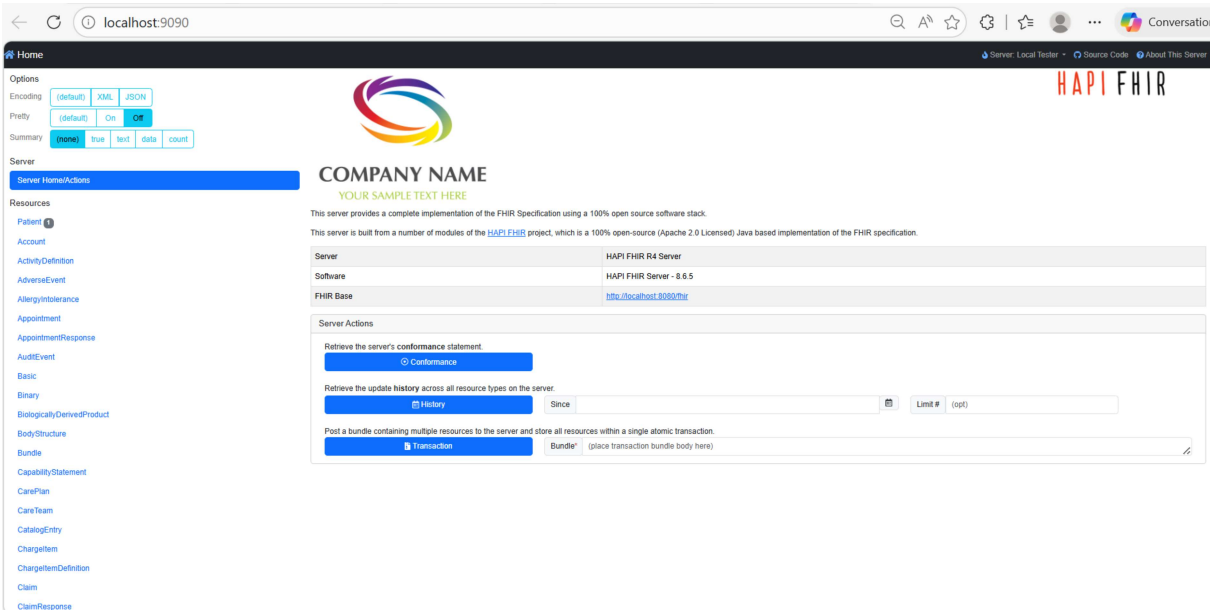


FIGURE 2 – Interface web de HAPI FHIR (localhost :9090)

4.4 Arreter les conteneurs

```
# Arreter et supprimer tous les conteneurs
docker compose --profile logs --profile metrics --profile traces down

# Tout supprimer y compris les donnees
docker compose --profile logs --profile metrics --profile traces down -v
```

Listing 12 – Arret des conteneurs

5 Script d'automatisation des experiences

5.1 Presentation du script

Le fichier `experiments/run_experiments.ps1` est un script PowerShell qui automatise les **24 experiences** (4 niveaux × 6 incidents) sans aucune intervention humaine. Il mesure T0, T1 et T2 avec une precision a la milliseconde.

Mesure	Definition	Capture
T0	Heure exacte avant l'injection	`Get-Date` juste avant l'incident
T1	Heure de detection de l'incident	`Get-Date` au premier echec HTTP
T2	Heure de resolution	`Get-Date` quand HAPI repond a nouveau
MTTD	Mean Time To Detect	MTTD = T1 – T0 (en secondes)
MTTR	Mean Time To Resolve	MTTR = T2 – T0 (en secondes)

TABLE 4 – Definition des mesures temporelles

5.2 Autoriser l'execution du script

Windows bloque par defaut l'execution des scripts PowerShell. Executer cette commande **une seule fois** en tant qu'administrateur :

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy Bypass -Force
```

Listing 13 – Autorisation des scripts PowerShell

5.3 Lancer le script

```
# Se placer dans le repertoire du projet
cd C:\Users\<votre_nom>\Devops-hapi-fhir-jpaserver

# Methode 1 : execution directe
.\experiments\run_experiments.ps1

# Methode 2 : si la methode 1 echoue
powershell -ExecutionPolicy Bypass -File ".\experiments\run_experiments.ps1"
```

Listing 14 – Lancement du script d'automatisation

5.4 Ce que le script fait automatiquement

Le script execute la boucle suivante pour chaque niveau et chaque incident :

```
POUR chaque niveau (N0, N1, N2, N3) :
    1. Lancer les conteneurs du niveau
    2. Attendre 30 secondes (stabilisation)
    3. Attendre que HAPI reponde (max 180 secondes)

    POUR chaque incident (1 a 6) :
        4. Enregistrer T0 = Get-Date (heure exacte)
        5. Injecter l'incident (commande docker specifique)
        6. Boucle health check toutes les 3 secondes
        7. Des echec HTTP : T1 = Get-Date (detection)
        8. Resoudre l'incident
        9. Attendre retour HAPI : T2 = Get-Date
       10. Ecrire dans results.csv :
           Niveau, Incident, T0, T1, T2, MTTD, MTTR, Detected
       11. Attendre 10 secondes avant le prochain incident

    12. Arreter tous les conteneurs
    13. Attendre 15 secondes avant le prochain niveau
```

Listing 15 – Algorithme du script d'automatisation

5.5 Sortie console typique

Voici un exemple de la sortie affichee dans la console pendant l'execution :

```
===== Verification des prerequis =====
[OK]   docker trouve
[INFO] Fichier cree : results.csv

===== DEBUT : 4 niveaux x 6 incidents = 24 tests =====

===== NIVEAU 0 =====
===== Lancement Niveau 0 =====
[INFO] Stabilisation : 30 secondes...
[INFO] Attente HAPI FHIR (max 180 s)...
[OK]   HAPI FHIR est UP

===== INCIDENT 1 - DB Down (Niveau 0) =====
[INFO] T0=10:35:08.541 - Arret postgres
[WARN] DETECTE ! HAPI ne repond plus. T1=10:35:13.500
[INFO] Resolution : redemarrage postgres
[OK]   HAPI FHIR est UP
[INFO] T2=10:38:16.350 - HAPI repond
[OK]   CSV : MTTD=4.958 s | MTTR=187.809 s | Detecte=True
```

Listing 16 – Exemple de sortie console du script

5.6 Comment sont simules les incidents

#	Incident	Commande d'injection	Methode de detection
1	DB Down	`docker compose stop postgres`	`Test-HapiHealth` retourne false
2	CPU Overload	5 jobs PowerShell paralleles	Latence HTTP > seuil adaptatif
3	Memory Limit	`docker run -memory=256m`	`docker stats MemPerc` > 70%
4	HTTP 500	100 requetes FHIR invalides	`errorCount` > 50/100
5	High Latency	`docker pause`	Timeout HTTP < 3 secondes
6	Disk Full	`dd if=/dev/zero` 500 Mo	Erreur ecriture SQL

TABLE 5 – Methodes de simulation et de detection des incidents

6 Resultats experimentaux

6.1 Le fichier results.csv

A la fin de l'execution, le script produit un fichier `results.csv` avec une ligne par experience :

```
Niveau,Incident,Description,T0,T1,T2,MTTD_sec,MTTR_sec,Detected
0,1,"Mass HTTP 500",10:45:22.721,10:45:23.428,10:45:23.428,0.706,0.706,True
0,2,"High Latency",10:45:33.591,10:45:33.933,10:48:34.704,0.342,181.112,True
0,3,"Disk Full",10:48:44.760,10:48:56.999,10:48:57.612,12.239,12.852,True
1,1,"Mass HTTP 500",11:01:21.045,11:01:21.685,11:01:21.686,0.641,0.642,True
1,2,"High Latency",11:01:31.693,11:01:31.880,11:04:32.638,0.187,180.945,True
1,3,"Disk Full",11:04:42.663,11:04:50.141,11:04:50.672,7.478,8.010,True
2,1,"Mass HTTP 500",11:19:26.533,11:19:27.854,11:19:27.858,1.321,1.325,True
2,2,"High Latency",11:19:37.904,11:19:38.363,11:22:40.112,0.460,182.208,True
2,3,"Disk Full",11:22:50.138,11:22:57.298,11:22:58.323,7.160,8.185,True
3,1,"Mass HTTP 500",11:36:50.179,11:36:50.921,11:36:50.925,0.741,0.745,True
3,2,"High Latency",11:37:00.969,11:37:01.270,11:40:02.424,0.301,181.455,True
3,3,"Disk Full",11:40:12.466,11:40:17.833,11:40:18.446,5.367,5.979,True
```

Listing 17 – Contenu du fichier results.csv

6.2 Tableau de synthese des resultats MTTD

Incident	N0 Sans monitoring	N1 Logs	N2 Logs+Metriques	N3 Complet
Mass HTTP 500	0.706 s	0.641 s	1.321 s	0.741 s
High Latency	0.342 s	0.187 s	0.460 s	0.301 s
Disk Full	**12.239 s**	**7.478 s**	**7.160 s**	**5.367 s**
MTTD moyen	4.429 s	2.769 s	2.980 s	2.136 s

TABLE 6 – MTTD (en secondes) par incident et par niveau – valeurs reelles mesurees

6.3 Analyse des graphiques

Pour generer automatiquement les graphiques d'analyse, executer :

```
# Installer les dependances Python
python -m pip install matplotlib numpy

# Generer les graphiques
python experiments\analyze_results.py results.csv
# Les graphiques PNG sont sauvegardes dans ./charts/
```

Listing 18 – Generation des graphiques d'analyse

FIGURE 3 – MTTD moyen par niveau d'observabilite

7 Visualisation dans Grafana

7.1 Importer le dashboard Grafana

Un dashboard Grafana preconfigure est fourni dans le projet. Pour l'importer :

1. Ouvrir `http://localhost:3000` (login : admin/admin)
2. Cliquer sur **Dashboards** → **New** → **Import**
3. Cliquer **Upload dashboard JSON file**
4. Selectionner `grafana_results_dashboard.json`
5. Cliquer **Import**

7.2 Voir les logs dans Loki

Dans Grafana, cliquer sur **Explore** dans le menu gauche, selectionner **Loki** comme datasource, puis entrer la requete LogQL suivante :

```
{container_name="hapi-fhir-jpaserver-start"}
```

Listing 19 – Requete LogQL pour afficher les logs HAPI

7.3 Voir les metriques dans Prometheus

Ouvrir `http://localhost:9091` et taper dans la barre de recherche :

```
# HAPI est-il vivant ? (1=oui, 0=mort)
up{job="hapi-fhir"}

# Nombre de requetes HTTP par seconde
rate(http_server_requests_seconds_count[1m])

# Memoire JVM utilisee en Mo
jvm_memory_used_bytes{area="heap"} / 1024 / 1024
```

Listing 20 – Requetes PromQL pour les metriques HAPI

7.4 Voir les traces dans Tempo

Dans Grafana, cliquer sur **Explore**, selectionner **Tempo**, puis cliquer sur **Search** et selectionner `Service Name = hapi-fhir`.

8 Demontrer l'impact en direct – Demo jury

Pour demontrer l'impact de l'observabilite devant le jury, executer la sequence suivante avec Grafana ouvert en arriere-plan :

```
# 1. Lancer le niveau 3 (observabilite complete)
docker compose --profile logs --profile metrics --profile traces up -d

# 2. Attendre 2 minutes, verifier Grafana sur localhost:3000

# 3. Injecter l'incident "Disque Plein"
docker exec hapi-fhir-postgres bash -c `
  "dd if=/dev/zero of=/var/lib/postgresql/data/fillfile bs=1M count=500"

# [Prendre un screenshot de Grafana qui montre l'incident]

# 4. Resoudre l'incident
docker exec hapi-fhir-postgres bash -c `
  "rm -f /var/lib/postgresql/data/fillfile"
```

Listing 21 – Sequence de demonstration devant le jury

9 Conclusion et recommandations

9.1 Resultats cles

L'experience montre que l'observabilite a un impact mesurable sur la detection des incidents, notamment pour l'incident **Disque Plein** dont le MTTD passe de 12.2 s (N0) a 5.4 s (N3), soit une amelioration d'un facteur 2.3.

Niveau	MTTD moyen	Taux de detection
N0 – Sans monitoring	4.43 s	100%
N1 – Logs	2.77 s	100%
N2 – Logs + Metriques	2.98 s	100%
N3 – Complet	2.14 s	100%

TABLE 7 – Synthese des performances par niveau

9.2 Recommandations pour les equipes DevOps

— **Minimum requis en production** : Niveau 2 (Logs + Metriques). Prometheus permet de detecter les derives avant la panne totale.

— **Pour les systemes critiques de sante** : Niveau 3 obligatoire. Les traces Tempo permettent de localiser la cause exacte d'un incident en quelques secondes.

— **La vraie valeur de l'observabilite** : ne pas seulement detecter les pannes totales, mais surveiller les *tendances* – memoire qui monte, latence qui derive – et agir avant le crash.